

Technical Documentation: Retail Data Pipeline & BI Solution

Architecture: From Normalized OLTP to Analytical Star Schema

1. Architectural Overview

The core objective was to engineer a scalable data infrastructure capable of converting raw transactional records into high-performance analytical structures. The pipeline is architected into three distinct layers:

- **Source:** Raw ingestion of transactional datasets (500k+ records).
- **Operational Layer (Retail_OLTP):** A 3rd Normal Form (3NF) relational model designed for ACID compliance and data integrity.
- **Analytical Layer (Retail_DWH):** A denormalized Star Schema optimized for OLAP workloads and complex business reporting.

2. Schema Design & Data Modeling

A) Operational Database (OLTP)

The schema was designed to eliminate redundancy and enforce referential integrity:

- **Master Data Management:** Centralized entities for Stores, Employees, and Customers.
- **Transaction Modeling:** A Master-Detail pattern (SalesHeaders/SalesDetails) to decouple invoice-level metadata from line-item granularity.

B) Data Warehouse (DWH)

Data was transformed into a multi-dimensional Star Schema to minimize join complexity:

- **Fact Table (Sales_Fact):** Consolidates metrics (Quantity, UnitPrice, TotalPrice) with Surrogate Keys for performance.
- **Dimensions:** Five-axis analysis (Time, Product, Customer, Store, Employee) for granular slicing and dicing of retail KPIs.

3. Engineering Challenges & Technical Solutions

Challenge 1: Heuristic Data Categorization

- **Issue:** Missing product taxonomies in the source data.
- **Implementation:** Developed a Python-based mapping engine. Using regex and keyword matching on the Description field, products were programmatically assigned to logical business categories.

Challenge 2: ODBC Driver & Type Handling

- **Issue:** Encountered HYC00 errors due to Python-SQL datetime incompatibility during bulk loads.

- **Implementation:** Standardized the ETL pipeline using `.strftime()` for ISO-compliant string conversion. Extracted the Hour component as a separate feature for peak-time sales analysis.

Challenge 3: Historical Data Integrity (SCD Type 2)

- **Issue:** Dynamic price fluctuations in the source system risked distorting historical revenue reports.
- **Implementation:** Deployed **Slowly Changing Dimension (SCD) Type 2** on the `Product_Dim` table. Implemented `IsActive`, `Start_Date`, and `End_Date` flags to preserve a full audit trail of price versions.

Challenge 4: Query Performance Optimization

- **Issue:** Real-time calculation of `TotalPrice` across 500k+ rows caused significant latency.
- **Implementation:** Optimized the schema with **Persisted Computed Columns**. By physically storing the calculated price on disk, read performance was improved by orders of magnitude.

4. Data Governance & SCD Strategy

- **SCD Type 1:** Used for Store and Employee dimensions where historical tracking of metadata (e.g., store name changes) was not required for this use case.
- **SCD Type 2:** Strictly enforced for Product and Customer dimensions to ensure financial accuracy and longitudinal behavioral analysis.

5. Key Highlights & Scalability

- **Automated Pipeline:** An end-to-end ETL workflow using Python (Pandas & PyODBC) for automated batch processing.
- **BI-Ready:** Engineered to support complex queries, such as regional profitability analysis by hour, in sub-second response times.
- **Future Proofing:** The pipeline is designed to decouple data cleaning (Python) from ingestion (SQL Bulk Inserts) to mitigate database locks as volume scales.

Technical Stack: SQL Server (TSQL), Python 3.x, Pandas, PyODBC.